

Understanding Mathematical Concepts via Go Language

Abstraction

Hyosang Kang (DGIST)

고 언어로 수학적 개념 이해하기

추상화

강효상 (DGIST)

Abstraction

What is Abstraction?

- In Mathematics, **abstraction** is the process of extracting the core properties (or axioms) of an object.
- In Programming, **abstraction** is the process of masquerading the details of an object by providing a set of methods.

Abstraction in Mathematics (1)

Notion of **set**

- Generally, a notion of **set** is defined to be a *collection of objects*.
- The meaning of terms *collection* and *object* are vague.
 - Russell's paradox: "Does the set of all elements that do not contain itself as a member contain itself?"
 - Mathematics does not allow contradiction such as "This statement is false."
- The **set** is defined by a list of **axioms**.
 - For example, the **axiom of extensionality** states that two sets are equal if they have the same elements.
 - The **axiom of union** states that the union of a set of sets is a set.

Abstraction in Mathematics (2)

Notion of Natural number

- Natural number is defined by its *counting* property.
 - Natural numbers must be *distinguishable*.
 - Natural numbers have *ordering*.
- Natural number are the first numbers to be defined. So we must start from *nothing*, i.e. the empty set $\emptyset$.

$$0 = \emptyset$$

$$1 = \{0\}$$

$$2 = \{0, 1\}$$

$$3 = \{0, 1, 2\}$$

Abstraction in Mathematics (3)

Notion of Real number

- Real number is defined by its axioms.
 - **Addition:** $x + y = y + x$
 - **Multiplication:** $x \cdot y = y \cdot x$
 - **Ordering:** $x < y$
 - **Completeness:** Every Cauchy sequence converges.
- Anything that satisfies these axioms can be treated as a **real number**.

How does abstraction works in Programming?

- In programming, every data is represented by a **type**.
- In some programming languages, there are notions of **abstract type**, which is a type defined by a set of **methods**.
- The user can use the object without knowing the details of the object.
- Such abstraction allows the user to focus on the *usage* of the object rather than the *implementation* of the object.

Abstraction in Programming

- An **element** is an object that one can compare it with another to see if they are equal.

```
type Element interface {  
    Equals(any) bool  
}
```

- A **finite(ness)** is a property that an object *contains* a finite number of elements, so one can count the number of elements.

```
type Finite interface {  
    Len() int  
}
```

Abstractions in Mathematics and Programming

"It is not who you are underneath, it is what you do that defines you." - Batman Begins

Mathematics	Programming
objects	types*
axioms	methods
invariants	function**

*or abstract types

**or the output(data, file, etc) of a function

This video was produced with the support of CTL in DGIST.

Production: Hyosang Kang

Editing: Jongwoo Baek

본 동영상은 DGIST의 CTL의 지원을 받아 제작되었습니다.

제작: 강효상

편집: 백종우